

WEB APPLICATION SECURITY

1 — QUICK DEFINITIONS (SHORT)

* **STORED XSS (PERSISTENT):** ATTACKER SUBMITS DATA (COMMENT, POST, PROFILE) THAT IS SAVED BY THE APP AND LATER SERVED TO OTHER USERS; THE INJECTED SCRIPT RUNS WHEN VICTIMS VIEW THE STORED CONTENT.

([PORTSWIGGER][1])

* **REFLECTED XSS (NON-PERSISTENT):** ATTACKER CRAFTS A URL OR FORM THAT INCLUDES SCRIPT IN A PARAMETER; THE APP REFLECTS IT BACK IN THE RESPONSE AND THE VICTIM'S BROWSER EXECUTES IT WHEN THEY OPEN THE LINK.

([PORTSWIGGER][2])

2 — DVWA: HOW TO DEMONSTRATE (LAB-ONLY, BENIGN)

DVWA EXPOSES XSS PAGES (LOW/MEDIUM/HIGH). TYPICAL FLOWS:

****STORED XSS (DVWA COMMENT BOX)****

1. OPEN DVWA → "STORED XSS" PAGE.

2. IN THE COMMENT FIELD PASTE A BENIGN PAYLOAD (PROOF-OF-CONCEPT):

```
```HTML
```

```
<SCRIPT>ALERT('DVWA-STORED-XSS')</SCRIPT>
```

```
```
```

3. SUBMIT. THE COMMENT IS STORED AND LATER DISPLAYED; WHEN A USER (OR YOU) VIEWS THE PAGE THE BROWSER EXECUTES THAT SCRIPT — YOU SHOULD SEE THE ALERT. THIS DEMONSTRATES PERSISTENCE AND IMPACT IN THE CONTROLLED LAB. ([COMPUTERSECURITYSTUDENT.COM])[3])

****REFLECTED XSS VIA QUERY PARAMETER (LAB-ONLY)****

IF DVWA (OR ANY TEST PAGE) ECHOES A `Q` PARAMETER INTO THE PAGE WITHOUT ENCODING, CRAFT A URL LIKE:

...

HTTP://<DVWA>/VULNERABILITIES/XSS_R/?NAME=<SCRIPT>ALERT('REFLECTED')</SCRIPT>

...

WHEN YOU OPEN THAT URL IN THE LAB, THE SCRIPT EXECUTES BECAUSE THE APP REFLECTED UNSANITIZED INPUT. (USE A BROWSER ON YOUR ISOLATED LAB.) ([MEDIUM])[4])

> THESE ARE CLASSIC, INTENTIONALLY-VULNERABLE BEHAVIOURS — USED FOR LEARNING AND DEFENSES. DO NOT RUN AGAINST EXTERNAL SITES. ([OWASP FOUNDATION])[5])

3 — WHY THIS HAPPENS (SHORT TECHNICAL CAUSE)

* APP READS USER INPUT (POST/QUERY), THEN INSERTS IT INTO HTML OUTPUT WITHOUT PROPERLY ****ENCODING/ESCAPING**** OR VALIDATING IT. BROWSERS EXECUTE THE INJECTED ``<SCRIPT>`` AS PAGE JS. FIXING REQUIRES TREATING ALL

UNTRUSTED DATA AS UNSAFE AND ESCAPING AT OUTPUT TIME. ([OWASP FOUNDATION])[5])

4 — MITIGATION 1 — OUTPUT ENCODING (RECOMMENDED PRIMARY FIX)

ALWAYS ENCODE DATA AT THE POINT OF OUTPUT FOR THE CONTEXT (HTML BODY, ATTRIBUTE, JS STRING, URL, CSS).

****PHP EXAMPLE (ESCAPE FOR HTML):****

```
` `` PHP
// WHEN DISPLAYING USER INPUT IN HTML
ECHO htmlspecialchars($USER_INPUT, ENT_QUOTES | ENT_HTML5, 'UTF-8');
` ``
```

THIS CONVERTS `<`, `>`, `"` AND `'```` TO HTML ENTITIES SO INJECTED SCRIPTS RENDER AS TEXT, NOT EXECUTABLE CODE. USE FRAMEWORK TEMPLATING/ESCAPING HELPERS WHERE AVAILABLE (THEY DO THIS FOR YOU). ([MDN WEB DOCS])[6])

****GENERAL RULES****

* ESCAPE FOR THE ****SPECIFIC**** CONTEXT (HTML ELEMENT, ATTRIBUTE, JAVASCRIPT CONTEXT, URL, CSS).

* PREFER OUTPUT ENCODING OVER NAIVE INPUT STRIPPING; ENCODING IS ROBUST EVEN IF ATTACKERS TRY NOVEL PAYLOADS. ([OWASP FOUNDATION])[5])

5 — MITIGATION 2 — INPUT VALIDATION (DEFENSE IN DEPTH)

* APPLY **WHITELISTING** WHERE POSSIBLE (ALLOW ONLY EXPECTED CHARACTERS/FORMAT). EXAMPLE: USERNAMES SHOULD MATCH ``^[A-ZA-Z0-9_]{3,30}$``.

* FOR FREE-FORM CONTENT (COMMENTS), DON'T TRY TO ALLOW/CLEAN HTML; INSTEAD EITHER:

* DISALLOW HTML (STRIP TAGS) AND THEN ESCAPE OUTPUT, **OR**

* IF YOU MUST ALLOW RICH TEXT, USE A SAFE SANITIZER LIBRARY (E.G., HTML PURIFIER FOR PHP) THAT PRODUCES SAFE HTML.

* NEVER RELY SOLELY ON CLIENT-SIDE VALIDATION. ALWAYS VALIDATE SERVER-SIDE. ([OWASP FOUNDATION][5])

EXAMPLE (PHP SIMPLE WHITELIST):

``` PHP

```
IF (!PREG_MATCH('/^[A-ZA-Z0-9_]{3,30}$/', $USERNAME)) {
 DIE('INVALID USERNAME');
}
```

```

6 — MITIGATION 3 — CONTENT SECURITY POLICY (CSP) — PRACTICAL EXAMPLES

CSP IS A BROWSER-ENFORCED HEADER THAT RESTRICTS WHERE SCRIPTS/STYLES/IMAGES CAN LOAD FROM; IT ****REDUCES**** THE IMPACT OF XSS BY MAKING INJECTED INLINE SCRIPTS FAIL (IF PROPERLY CONFIGURED). CSP IS ***DEFENSE IN DEPTH*** — IT DOESN'T FIX THE UNDERLYING BUG BUT MAKES EXPLOITATION HARDER. ([WEB.DEV][7])

****SIMPLE STRONG CSP (HEADER EXAMPLE):****

...

```
CONTENT-SECURITY-POLICY: DEFAULT-SRC 'SELF'; SCRIPT-SRC 'SELF'; OBJECT-SRC 'NONE'; BASE-URI 'SELF';
```

...

THIS ALLOWS RESOURCES ONLY FROM THE SAME ORIGIN AND BLOCKS INLINE SCRIPTS.

****BETTER (NONCE-BASED) — ALLOWS ONLY SCRIPTS WITH A SERVER-GENERATED NONCE:****

1. SERVER GENERATES A RANDOM NONCE PER RESPONSE AND ADDS IT TO ALLOWED SCRIPT SOURCES:

...

```
CONTENT-SECURITY-POLICY: SCRIPT-SRC 'NONCE-<RANDOMBASE64>' 'STRICT-DYNAMIC' 'UNSAFE-INLINE' 'UNSAFE-EVAL';
```

...

2. SERVER INJECTS THE SAME NONCE INTO TRUSTED ``<SCRIPT NONCE="...">`` TAGS. USING NONCES/HASHES IS RECOMMENDED FOR COMPLEX APPS. ([MDN WEB DOCS][6])

****HOW TO SET CSP IN APACHE (.HTACCESS):****

```APACHE

```
HEADER SET CONTENT-SECURITY-POLICY "DEFAULT-SRC 'SELF'; SCRIPT-SRC 'SELF';
OBJECT-SRC 'NONE';"
```

```

****IMPORTANT CSP CAVEATS****

* AVOID ``UNSAFE-INLINE`` — IT WEAKENS CSP.

* CSP CAN BE BYPASSED IF THE PAGE ALREADY CONTAINS TRUSTED SCRIPT SOURCES THAT ACCEPT ATTACKER-CONTROLLED DATA, SO COMBINE CSP WITH PROPER ENCODING. ([PORTSWIGGER][8])

7 — EXTRA DEFENSES & BEST PRACTICES (SHORT LIST)

* USE ``HTTPONLY`` + ``SECURE`` COOKIES TO REDUCE COOKIE THEFT VIA XSS.

* APPLY ``SAMESITE`` COOKIE ATTRIBUTE.

* USE CSP + STRICT OUTPUT ENCODING + INPUT VALIDATION = GOOD DEFENSE-IN-DEPTH.

* USE A WEB APPLICATION FIREWALL (WAF) AND REGULAR CODE/SECURITY REVIEWS. ([OWASP CHEAT SHEET SERIES][9])

8 — QUICK CHECKLIST FOR HARDENING DVWA/DEMO APPS

1. TURN DVWA SECURITY UP (MEDIUM/HIGH) TO SEE HOW MITIGATIONS CHANGE BEHAVIOUR.

2. APPLY SERVER-SIDE ESCAPING (E.G., `HTMLESPECIALCHARS` IN PHP) WHERE DVWA DISPLAYS USER DATA. TEST AGAIN — ALERTS SHOULD NO LONGER RUN. ([COMPUTERSECURITYSTUDENT.COM][3])

3. ADD A CSP HEADER AND VERIFY INJECTED INLINE SCRIPTS ARE BLOCKED BY THE BROWSER CONSOLE.

4. USE AN HTML SANITIZER LIKE HTML PURIFIER IF YOU MUST ACCEPT LIMITED HTML FROM USERS.

9 — REFERENCES & FURTHER READING

* PORTSWIGGER — STORED & REFLECTED XSS EXPLANATIONS AND TESTING GUIDANCE. ([PORTSWIGGER][1])

* MDN / WEB.DEV — CSP GUIDES AND BEST PRACTICES. ([MDN WEB DOCS][6])

* OWASP XSS DOCUMENTATION & CSP CHEAT SHEET. ([OWASP FOUNDATION][5])

IF YOU WANT I CAN:

* PRODUCE A STEP-BY-STEP ****DVWA PATCH**** (SAFE CODE SNIPPET) THAT REPLACES THE VULNERABLE OUTPUT WITH ``HTMLSPECIALCHARS()`` CALLS AND SHOWS THE EXACT FILE(S) TO EDIT FOR YOUR DVWA INSTALL — READY TO PASTE INTO YOUR LAB.

* OR, ANALYZE A DVWA PAGE YOU PASTE (HTML/PHP SNIPPET) AND SHOW THE MINIMAL CHANGES TO MITIGATE XSS.

I CAN WALK YOU THROUGH **SAFE, LAB-ONLY** EXAMPLES SHOWING HOW CSRF, LFI AND RFI WORK IN A VULNERABLE ENVIRONMENT SUCH AS DVWA — AND HOW TO PROTECT AGAINST THEM. I'LL GIVE:

- A BENIGN CSRF DEMO (HTML PAGE THAT AUTO-SUBMITS) YOU CAN RUN **ONLY** AGAINST YOUR DVWA LAB,
- SERVER-SIDE TOKEN (PHP) PROTECTION TO BLOCK CSRF,
- LFI EXAMPLES THAT DEMONSTRATE READING LOCAL FILES IN A LAB AND SECURE FIXES,
- RFI EXPLANATION AND SECURE CONFIGURATION (BUT **NO** PAYLOADS TO UPLOAD/EXECUTE REMOTE CODE),
- MITIGATION CHECKLIST (PRACTICAL FIXES: TOKENS, SAMESITE, INPUT VALIDATION, CONFIG HARDENING).

STRONG SAFETY NOTE: DO **NOT** USE THE ATTACK EXAMPLES ON ANY SITE YOU DO NOT OWN OR EXPLICITLY HAVE WRITTEN PERMISSION TO TEST. ALL EXAMPLES BELOW ARE INTENDED FOR ISOLATED LAB TESTING (DVWA) ONLY.

1) CSRF — CONCEPT RECAP (ONE LINE)

CSRF TRICKS A LOGGED-IN USER'S BROWSER INTO MAKING AN UNWANTED REQUEST TO A SITE WHERE THEY'RE AUTHENTICATED (E.G., CHANGING PASSWORD) BY SUBMITTING A FORGED REQUEST (FORM/IMAGE/LINK) FROM ANOTHER SITE.

2) BENIGN CSRF DEMO (LAB-ONLY — REFLECTED AGAINST DVWA)

IF DVWA HAS A PASSWORD-CHANGE ENDPOINT LIKE `POST /VULNERABILITIES/CSRF/` WITH FIELDS `PASSWORD_NEW` AND `PASSWORD_CONF`, HERE'S A **BENIGN** HTML PROOF-OF-CONCEPT PAGE YOU CAN HOST ON YOUR ATTACKER VM. IT AUTO-

SUBMITS A POST THAT WILL ATTEMPT TO CHANGE THE DVWA USER'S PASSWORD **ONLY IF** THE VICTIM IS LOGGED INTO DVWA IN THE SAME BROWSER.

SAVE AS CSRF_DEMO.HTML ON YOUR ATTACKER VM AND OPEN IT IN THE BROWSER THAT HAS A DVWA SESSION TO OBSERVE THE EFFECT (LAB-ONLY):

```
<!DOCTYPE HTML>

<HTML>

  <HEAD><META CHARSET="UTF-8"><TITLE>CSRF DEMO (LAB ONLY)</TITLE></HEAD>

  <BODY>

    <H3>CSRF DEMO — AUTO SUBMITTING A PASSWORD CHANGE (DVWA)</H3>


    <!-- REPLACE ACTION WITH THE EXACT DVWA FORM ACTION URL FOR PASSWORD
    CHANGE -->

    <FORM ID="CSRF" ACTION="HTTP://DVWA_HOST/VULNERABILITIES/CSRF/"
    METHOD="POST">

      <!-- FIELDS NAMED ACCORDING TO DVWA'S FORM (EXAMPLE NAMES) -->

      <INPUT TYPE="HIDDEN" NAME="PASSWORD_NEW"
      VALUE="NEWLABPASSWORD123" />

      <INPUT TYPE="HIDDEN" NAME="PASSWORD_CONF"
      VALUE="NEWLABPASSWORD123" />

      <INPUT TYPE="HIDDEN" NAME="CHANGE" VALUE="CHANGE" />

    </FORM>


    <SCRIPT>

      // AUTO-SUBMIT AFTER SHORT DELAY SO YOU CAN OBSERVE

      SETTIMEOUT(FUNCTION(){ DOCUMENT.GETELEMENTBYID('CSRF').SUBMIT(); },
      1500);

    </SCRIPT>

  </BODY>
```

</HTML>

WHAT HAPPENS IN THE LAB:

- IF THE BROWSER HAS A VALID DVWA SESSION COOKIE, DVWA RECEIVES THE POST AND (ON A VULNERABLE CONFIG) CHANGES THE PASSWORD.
- IF DVWA USES CSRF TOKENS OR OTHER PROTECTIONS, THE REQUEST SHOULD FAIL.

3) TOKEN-BASED PROTECTION (SERVER-SIDE) — HOW TO IMPLEMENT AND TEST

BELOW IS A **SIMPLE** AND CLEAR PHP IMPLEMENTATION PATTERN (GENERATE TOKEN IN SESSION, EMBED TOKEN IN FORMS, VERIFY ON SUBMIT). THIS IS WHAT YOU SHOULD ADD TO THE DVWA PASSWORD-CHANGE HANDLING CODE (OR ANY FORM HANDLER).

A) GENERATE AND EMBED TOKEN (WHEN RENDERING THE FORM)

```
<?PHP
```

```
SESSION_START();
```

```
// CREATE TOKEN IF MISSING
```

```
IF (EMPTY($_SESSION['CSRF_TOKEN'])) {
```

```
    $_SESSION['CSRF_TOKEN'] = BIN2HEX(RANDOM_BYTES(32));
```

```
}
```

```
$TOKEN = $_SESSION['CSRF_TOKEN'];
```

```
?>
```

```
<FORM METHOD="POST" ACTION="/VULNERABILITIES/CSRF/">
```

```
    <INPUT TYPE="PASSWORD" NAME="PASSWORD_NEW" />
```

```
    <INPUT TYPE="PASSWORD" NAME="PASSWORD_CONF" />
```

```
    <INPUT TYPE="HIDDEN" NAME="CSRF_TOKEN" VALUE="<?PHP ECHO  
HTMLSPECIALCHARS($TOKEN, ENT_QUOTES, 'UTF-8'); ?>" />
```

```
    <INPUT TYPE="SUBMIT" VALUE="CHANGE" />
```

```
</FORM>
```

B) VERIFY TOKEN ON SUBMIT (SERVER-SIDE HANDLER)

```
<?PHP
```

```
SESSION_START();
```

```
IF ($_SERVER['REQUEST_METHOD'] === 'POST') {
```

```
    IF (EMPTY($_POST['CSRF_TOKEN']) || EMPTY($_SESSION['CSRF_TOKEN']))
```

```
        || !HASH_EQUALS($_SESSION['CSRF_TOKEN'], $_POST['CSRF_TOKEN'])) {
```

```
        // CSRF VALIDATION FAILED
```

```
        HTTP_RESPONSE_CODE(403);
```

```
        ECHO "CSRF TOKEN MISMATCH. REQUEST DENIED.";
```

```
        EXIT;
```

```
    }
```

```
    // OPTIONALLY, ROTATE TOKEN AFTER SUCCESSFUL USE:
```

```
    UNSET($_SESSION['CSRF_TOKEN']);
```

```
    // PROCEED TO VALIDATE PASSWORD FIELDS, UPDATE DB, ETC.
```

```
    $PW = $_POST['PASSWORD_NEW'] ?? '';
```

```
    $PW_CONF = $_POST['PASSWORD_CONF'] ?? '';
```

```
    // DO SERVER-SIDE VALIDATION, HASHING, AND UPDATE
```

```
}
```

```
?>
```

WHY IT WORKS: THE ATTACKER PAGE CANNOT READ OR GUESS THE SERVER-SIDE SESSION TOKEN (SAME-ORIGIN POLICY), SO THEY CANNOT CRAFT A CORRECT POST WITH A SERVER-ACCEPTED TOKEN.

4) ADDITIONAL CSRF DEFENSES (DEFENSE-IN-DEPTH)

- USE SAMESITE COOKIE ATTRIBUTE: SET-COOKIE: PHPSESSID=...; SAMESITE=STRICT; SECURE; HTTPONLY — PREVENTS THE BROWSER FROM SENDING SESSION COOKIES ON CROSS-SITE REQUESTS (WORKS WELL FOR MANY FLOWS).
 - VALIDATE ORIGIN OR REFERER HEADERS FOR SENSITIVE STATE-CHANGING REQUESTS (NOT SOLE PROTECTION, BUT USEFUL).
 - DOUBLE-SUBMIT COOKIE: SET A COOKIE WITH A RANDOM VALUE AND REQUIRE THE SAME VALUE IN A HIDDEN FORM FIELD; VERIFY THEY MATCH ON SERVER.
 - REQUIRE RE-AUTHENTICATION FOR CRITICAL ACTIONS (ENTER CURRENT PASSWORD BEFORE CHANGING TO A NEW ONE).
 - KEEP CSRF TOKENS PER-FORM AND ROTATE THEM, SET EXPIRATION.
-

5) LOCAL FILE INCLUSION (LFI) — DEMONSTRATION AND SAFE TEST

VULNERABLE PATTERN (DO NOT USE IN PRODUCTION):

```
<?PHP
// UNSAFE: DIRECTLY USING USER INPUT IN INCLUDE()

$page = $_GET['PAGE'];

include($page); // VULNERABLE TO LFI

?>
```

A) LFI DEMO IN LAB (READING LOCAL FILES)

IF DVWA HAS AN LFI MODULE, TO TEST IN LAB YOU MIGHT CALL (FROM A BROWSER OR CURL):

```
HTTP://DVWA_HOST/VULNERABILITIES/FI/?PAGE=/ETC/PASSWD
```

IF VULNERABLE, THE SERVER WILL INCLUDE AND OUTPUT /ETC/PASSWD CONTENT. ONLY DO THIS ON YOUR OWN DVWA LAB.

B) SECURE FIXES FOR LFI

1. WHITELIST ALLOWED FILES:

```
$ALLOWED = [
```

```
'HOME' => '/VAR/WWW/HTML/PAGES/HOME.PHP',  
'ABOUT' => '/VAR/WWW/HTML/PAGES/ABOUT.PHP',  
];
```

```
$K = $_GET['PAGE'] ?? 'HOME';  
IF (!ARRAY_KEY_EXISTS($K, $ALLOWED)) {  
    // ERROR / 404  
    HTTP_RESPONSE_CODE(404);  
    EXIT;  
}  
INCLUDE $ALLOWED[$K];
```

2. **SANITIZE + CANONICALIZE:**

```
$PAGE = $_GET['PAGE'] ?? "";  
$CLEAN = BASENAME($PAGE); // STRIPS DIRECTORY TRAVERSAL LIKE ../..  
$PATH = "/VAR/WWW/HTML/PAGES/{$CLEAN}.PHP";  
$REAL = REALPATH($PATH);  
IF ($REAL === FALSE || STRPOS($REAL, '/VAR/WWW/HTML/PAGES/') !== 0) {  
    HTTP_RESPONSE_CODE(400); EXIT;  
}  
INCLUDE $REAL;
```

3. **AVOID DYNAMIC INCLUDE OF USER-PROVIDED PATHS** WHEREVER POSSIBLE.

WHY THIS PREVENTS LFI: WHITELISTING RESTRICTS WHICH FILES MAY BE INCLUDED; BASENAME/REALPATH PREVENTS DIRECTORY TRAVERSAL.

6) REMOTE FILE INCLUSION (RFI) — EXPLANATION, WHY DANGEROUS, AND SAFE HANDLING

RFI IS WHEN USER INPUT CONTROLS AN INCLUDE() THAT LOADS A REMOTE URL (E.G., INCLUDE(\$_GET['URL'])) — THIS CAN RESULT IN EXECUTING ATTACKER-CONTROLLED CODE ON THE SERVER (VERY DANGEROUS).

DANGEROUS PATTERN (DO NOT ENABLE):

// VERY DANGEROUS, DO NOT USE

INCLUDE(\$_GET['FILE']); // IF ALLOW_URL_INCLUDE=ON, THIS CAN INCLUDE REMOTE CODE

WHY RFI IS RISKY

- A REMOTE ATTACKER CAN HOST PHP CODE AND CAUSE THE SERVER TO EXECUTE IT.
- LEADS TO FULL SERVER COMPROMISE.

SECURE CONFIGURATION & MITIGATIONS (PRACTICAL)

- **DISABLE REMOTE INCLUDES IN PHP:**
 - ALLOW_URL_FOPEN = OFF (RECOMMENDED)
 - ALLOW_URL_INCLUDE = OFF (MUST BE OFF TO PREVENT INCLUDING URLS)
- **DO NOT USE USER INPUT AS AN INCLUDE PATH** — USE A WHITELIST AS SHOWN IN LFI FIXES.
- VALIDATE AND CANONICALIZE INPUTS; NEVER ALLOW PROTOCOL PREFIXES (HTTP://, HTTPS://, FTP://).
- RUN PHP WITH LEAST PRIVILEGE, SEPARATE VHOSTS, NON-PRIVILEGED USER ACCOUNTS.
- USE WEB APPLICATION FIREWALLS TO DETECT SUSPICIOUS INCLUDE-RELATED REQUESTS.

I WILL **NOT** PROVIDE CODE TO HOST OR EXECUTE REMOTE PAYLOADS. IF YOU WANT TO PRACTICE RFI **SAFELY** IN A LAB, CREATE A LOCAL FILE AND TEST ONLY LOCAL INCLUDES; NEVER FETCH OR EXECUTE CODE FROM OUTSIDE YOUR LAB.

7) EXAMPLE DVWA-HARDENING CHECKLIST (CSRF, LFI, RFI)

- CSRF

- ADD AND VALIDATE ANTI-CSRF TOKENS ON ALL STATE-CHANGING FORMS.
 - USE SAMESITE=STRICT OR LAX ON AUTHENTICATION COOKIES.
 - VALIDATE ORIGIN & REFERER FOR CRITICAL ENDPOINTS.
 - REQUIRE CURRENT-PASSWORD CONFIRMATION FOR SENSITIVE ACTIONS.
 - LFI / RFI
 - REPLACE DYNAMIC INCLUDES WITH WHITELIST PATTERN OR ROUTING MAP.
 - USE REALPATH() + PREFIX CHECKS TO PREVENT TRAVERSAL.
 - DISABLE ALLOW_URL_INCLUDE AND ALLOW_URL_FOPEN IN PHP.INI.
 - RUN WEB APP WITH MINIMAL PRIVILEGES; ISOLATE FILE STORAGE.
 - LOGGING & MONITORING
 - LOG CSRF FAILURES, SUSPICIOUS INCLUDE PARAMETERS, AND FREQUENT 4XX RESPONSES.
 - ALERT ON REPEATED ATTEMPTS WITH DIRECTORY-TRAVERSAL STRINGS (../).
 - CODE PRACTICES
 - USE FRAMEWORKS THAT ESCAPE OUTPUT AND PROVIDE SAFE ROUTING MODULES.
 - REVIEW 3RD-PARTY LIBRARIES FOR INSECURE INCLUDE PATTERNS.
-

8) TESTING GUIDANCE (WHAT TO RUN IN THE LAB)

- FOR CSRF: HOST THE CSRF_DEMO.HTML ATTACKER PAGE ON ATTACKER VM AND OPEN IT IN A BROWSER SESSION THAT HAS DVWA LOGGED-IN — CONFIRM TOKEN-PROTECTED FORM REJECTS IT.
- FOR LFI: TRY ?PAGE=../../../../ETC/PASSWD ON YOUR DVWA LFI ENDPOINT AND CONFIRM IT'S BLOCKED AFTER APPLYING FIXES.

- FOR RFI: ENSURE PHP.INI HAS ALLOW_URL_INCLUDE = OFF; TRY TO FETCH ?FILE=HTTP://ATTACKER/... — IT SHOULD FAIL.
-

SAFETY & RULES OF ENGAGEMENT (READ FIRST)

- ONLY TEST SYSTEMS YOU OWN OR HAVE EXPLICIT PERMISSION TO TEST (DVWA, METASPLOITABLE IN AN ISOLATED LAB).
 - DON'T BRUTE-FORCE REAL ACCOUNTS OR BYPASS MFA ON PRODUCTION SYSTEMS. BE MINDFUL OF RATE LIMITS/ACCOUNT LOCKOUTS.
 - KEEP TRAFFIC INSIDE AN ISOLATED LAB NETWORK SO YOU DON'T ACCIDENTALLY IMPACT OTHERS.
-

TOOLS YOU CAN USE

- **BURP SUITE** (COMMUNITY OR PRO). *NOTE:* BURP PRO MAKES INTRUDER FASTER AND EASIER; COMMUNITY IS USABLE FOR LEARNING BUT SLOWER.
 - **OWASP ZAP** (FREE) — HAS AN INTERCEPTING PROXY, REPEATER AND A FUZZER (ALTERNATIVE TO BURP INTRUDER).
 - BROWSER (FIREFOX RECOMMENDED) CONFIGURED TO PROXY THROUGH BURP/ZAP.
 - A TEST APP (DVWA / LOCAL WEB APP) WITH A LOGIN FORM.
-

1) CONFIGURE YOUR BROWSER PROXY

1. START BURP (DEFAULT LISTENS ON 127.0.0.1:8080).
 2. IN BROWSER (FIREFOX): PREFERENCES → NETWORK SETTINGS → MANUAL PROXY → HTTP PROXY 127.0.0.1 PORT 8080.
 3. INSTALL BURP CA CERT IN BROWSER (SO HTTPS WORKS) — ONLY IN LAB.
-

2) INTERCEPTING AND MODIFYING A LOGIN REQUEST (BURP)

1. IN BURP → PROXY → INTERCEPT → TURN **INTERCEPT ON**.

2. IN BROWSER, NAVIGATE TO THE LOGIN PAGE AND SUBMIT CREDENTIALS. BURP WILL STOP THE REQUEST.

3. REVIEW THE CAPTURED REQUEST IN BURP'S PROXY TAB:

- REQUEST LINE, HEADERS (COOKIE, REFERER, ORIGIN), BODY (FORM FIELDS).

4. **MODIFY FIELDS INLINE:**

- EDIT USERNAME=...&PASSWORD=... OR HEADERS, THEN CLICK **FORWARD** TO LET IT REACH THE SERVER.
- USE THIS TO TEST BEHAVIOR WHEN FIELDS ARE CHANGED, OR TO TEST PARAMETER TAMPERING (E.G., CHANGE ROLE=ADMIN IF PRESENT).

5. **SEND TO REPEATER** FOR CAREFUL MANUAL TESTING:

- RIGHT-CLICK → **SEND TO REPEATER**.
- IN REPEATER, YOU CAN EDIT THE REQUEST AND RESEND MANY TIMES, INSPECT RESPONSES, COMPARE STATUS CODES, RESPONSE LENGTHS AND BODY CONTENT.

USEFUL CHECKS:

- REMOVE OR CHANGE REFERER/ORIGIN TO TEST CSRF PROTECTIONS.
- MODIFY COOKIES OR SESSION TOKEN VALUES TO SEE HOW THE SERVER RESPONDS.
- CHANGE CONTENT-TYPE OR ENCODING TO TEST SERVER PARSING BEHAVIOR.

3) PREPARING FOR FUZZING WITH INTRUDER (BURP)

GOAL EXAMPLES: TEST PARAMETER HANDLING, USERNAME ENUMERATION, WEAK PASSWORD CHECKS, OR INPUT VALIDATION ISSUES.

TYPICAL INTRUDER USE CASES

- ENUMERATE VALID USERNAMES (KEEP LOW RATE; WATCH FOR LOCKOUT).
- TEST PASSWORD LISTS FOR A SINGLE USERNAME (IN LAB ONLY).
- FUZZ HIDDEN PARAMETERS OR HEADER VALUES TO FIND PARSING BUGS.

STEPS TO SET UP INTRUDER ATTACK

1. IN PROXY OR REPEATER, RIGHT-CLICK THE LOGIN REQUEST → **SEND TO INTRUDER**.

2. IN INTRUDER → **POSITIONS** TAB:

- BURP WILL AUTO-MARK MANY POSITIONS. CLICK **CLEAR S** TO REMOVE ALL, THEN MANUALLY MARK ONLY THE FIELDS YOU WANT TO FUZZ:
 - EXAMPLE: USERNAME=\$ADMIN\$&PASSWORD=\$PASSWORD123\$ — MARK USERNAME OR PASSWORD (OR BOTH FOR COMBINED ATTACKS).
- CHOOSE **ATTACK TYPE**:
 - **SNIPER** — SINGLE PAYLOAD POSITION, GOOD FOR DISCOVERING WHICH PAYLOAD TRIGGERS BEHAVIOR.
 - **BATTERING RAM** — SAME PAYLOAD AT ALL MARKED POSITIONS (RARE FOR LOGIN).
 - **PITCHFORK** — PARALLEL LISTS FOR POSITIONS (E.G., LIST OF USERNAMES AND MATCHING PASSWORDS).
 - **CLUSTER BOMB** — CARTESIAN PRODUCT (EVERY USERNAME × EVERY PASSWORD) — EXPENSIVE; USE ONLY IN LAB.

3. **OPTIONS** TAB:

- SET GREP - MATCH STRINGS OR USE RESPONSE COMPARISONS (LENGTH/STATUS/TIME) TO HIGHLIGHT INTERESTING RESPONSES. ADD STRINGS LIKE WELCOME, INVALID USERNAME, INCORRECT PASSWORD OR THE LOGIN SUCCESS DASHBOARD CONTENT.
- UNDER **ATTACK SPEED**, SET THROTTLING / DELAYS (IMPORTANT TO AVOID DOS OR LOCKOUTS). IN BURP PRO YOU CAN CONFIGURE CONCURRENCY; IN COMMUNITY, IT'S SLOWER BY DEFAULT.

4. **PAYLOADS** TAB:

- LOAD PAYLOAD LIST(S). BURP SUPPORTS PASTE, LOAD FROM FILE, OR BUILT-IN LISTS.
- USE SMALL TEST LISTS FIRST (E.G., ADMIN, TEST, GUEST) AND A FEW PASSWORDS (PASSWORD, 123456, P@SSW0RD) — KEEP IT CONSERVATIVE.

- USE PAYLOAD PROCESSING RULES (URL ENCODE, BASE64, ETC.) IF THE APP EXPECTS ENCODED INPUT.

5. **START ATTACK** (BURP PRO) — COMMUNITY USERS MUST MANUALLY CLICK EACH REQUEST OR USE REPEATER (OR USE A SMALL SCRIPTED APPROACH). IF YOU CAN'T START INTRUDER (COMMUNITY LIMITED), USE REPEATER WITH SMALL MANUAL LISTS.

4) DETECTING SUCCESSFUL OR INTERESTING RESPONSES

- **STATUS CODES:** 200 VS 302 (REDIRECT AFTER LOGIN SUCCESS IS COMMON), 401/403 FOR FAILURES.
- **RESPONSE LENGTH:** SUCCESS PAGE OFTEN HAS LARGER LENGTH — USE INTRUDER'S LENGTH SORT TO FIND OUTLIERS.
- **BODY CONTENT/GREP:** SET GREP KEYWORDS (E.G., WELCOME, DASHBOARD, LOGOUT) — INTRUDER WILL HIGHLIGHT MATCHES.
- **HEADERS:** SUCCESSFUL LOGIN MAY SET A NEW SET-COOKIE SESSION OR REDIRECT (LOCATION HEADER).
- **RESPONSE TIME:** SOME ENDPOINTS SHOW TIMING DIFFERENCES (BUT DANGEROUS TO RELY ON).

IN INTRUDER → **RESULTS**, SORT BY STATUS, LENGTH, OR THE GREP MATCH COLUMNS TO SPOT POSITIVES QUICKLY.

5) FUZZING STRATEGIES & PAYLOAD IDEAS (LAB-ONLY)

- **USERNAME ENUMERATION:** PUT CANDIDATE USERNAMES IN PAYLOAD LIST FOR USERNAME FIELD; SET PASSWORD TO A CONSTANT INVALID VALUE; LOOK FOR DIFFERING RESPONSES (E.G. "INVALID USERNAME" VS "INVALID PASSWORD").
- **PASSWORD FUZZING FOR SINGLE USERNAME:** SET USERNAME FIXED AND PAYLOADS = PASSWORD LIST. USE SMALL LISTS AND SLOW RATE.
- **PARAMETER TAMPERING:** FUZZ HIDDEN PARAMS (E.G., ROLE, IS_ADMIN) WITH BOOLEAN-ISH VALUES: 0,1,TRUE,FALSE,ADMIN,1% OR 1=1.

- **INJECTION-TESTING:** USE SMALL ENCODED PAYLOADS TO TEST FOR SQLI/XSS (E.G., ' OR '1'='1, <SCRIPT>ALERT(1)</SCRIPT>), BUT DO THIS ONLY IN LAB AND SANITIZE OUTPUTS.

EXAMPLE TINY PAYLOAD LISTS (SAFE FOR DEMO):

- USERNAMES.TXT:
- ADMIN
- TEST
- GUEST
- USER
- PASSWORDS.TXT:
- PASSWORD
- 123456
- SECRET123
- P@SSW0RD!

DO NOT USE MASSIVE PASSWORD LISTS OR HIGH-SPEED BRUTEFORCE AGAINST LIVE TARGETS.

6) PRACTICAL TIPS & DEFENSIVE AWARENESS

- ALWAYS THROTTLE THE ATTACK: SET DELAYS (E.G., 500–2000 MS) BETWEEN REQUESTS; USE SHORT LISTS FIRST.
 - WATCH FOR ACCOUNT LOCKOUT RULES AND MULTI-FACTOR AUTHENTICATION. RESPECT THEM.
 - CHECK SERVER LOGS (ON YOUR TEST APP) TO CORRELATE INTRUDER RESULTS WITH SERVER BEHAVIOR.
 - USE BURP -> LOGGER OR PROXY HISTORY TO TRACK WHAT YOU SENT AND THE RESPONSE HEADERS/COOKIES.
 - USE INTRUDER -> OPTIONS -> PAYLOAD PROCESSING TO ADD COMMON ENCODINGS (URL ENCODE, HTML ENCODE) THAT WEB APPS EXPECT.
-

7) ALTERNATIVE: OWASP ZAP FUZZER

IF YOU DON'T HAVE BURP PRO, ZAP'S FUZZER IS A POWERFUL FREE ALTERNATIVE:

1. INTERCEPT THE LOGIN REQUEST WITH ZAP PROXY.
 2. RIGHT-CLICK THE REQUEST/PARAMETER → **ATTACK** → **FUZZ....**
 3. SELECT THE PARAMETER(S) AND ADD PAYLOAD LISTS (ZAP SHIPS WITH BUILT-IN LISTS OR YOU CAN IMPORT).
 4. RUN WITH DELAYS AND WATCH RESULTS; CONFIGURE MATCHERS (STRINGS, REGEX, RESPONSE CODES) TO HIGHLIGHT INTERESTING RESPONSES.
-

8) EXAMPLE QUICK WORKFLOW (DVWA LOGIN) — STEP BY STEP

1. START BURP AND SET BROWSER PROXY.
 2. LOGIN TO DVWA NORMALLY IN YOUR BROWSER (SO YOU HAVE A SESSION).
 3. OPEN LOGIN PAGE AGAIN AND SUBMIT BOGUS CREDENTIALS WHILE BURP INTERCEPTS.
 4. SEND CAPTURED REQUEST TO INTRUDER. CLEAR POSITIONS, MARK USERNAME POSITION ONLY.
 5. LOAD USERNAMES.TXT AS PAYLOAD LIST. FIX PASSWORD FIELD TO A KNOWN BAD VALUE.
 6. CONFIGURE GREP FOR INVALID USERNAME AND INVALID PASSWORD. SET A 500 MS DELAY.
 7. LAUNCH ATTACK (OR RUN MANUALLY IF USING COMMUNITY). SORT RESULTS BY GREP/LENGTH TO FIND DIFFERENCES.
 8. INVESTIGATE ANY CANDIDATE USERNAMES IN REPEATER FOR CONFIRMATIONS.
-

9) LOGGING, EVIDENCE & REPORTING

- SAVE INTRUDER RESULTS (EXPORT) AND INCLUDE SAMPLE REQUESTS/RESPONSES IN YOUR REPORT.

- NOTE THE EXACT DETECTION CRITERIA YOU USED (E.G., RESPONSE LENGTH = 2482 → SUCCESS).
 - DOCUMENT RATE LIMITS AND ANY ACCOUNT LOCKOUTS YOU OBSERVED.
-

10) ETHICAL AND LEGAL REMINDERS

- DO NOT USE THESE TECHNIQUES AGAINST THIRD-PARTY SERVICES. EXPLAIN IN YOUR REPORT THAT TESTING WAS DONE IN AN ISOLATED LAB AND INCLUDE TIMESTAMPS, TARGET IPS, AND ATTACKER IPS.

TO ANALYZE A WEBSITE'S HTTP SECURITY POSTURE AND ADD PROPER HEADERS IN APACHE, USE **SECURITYHEADERS.COM** FOR SCANNING AND THEN UPDATE YOUR APACHE CONFIGURATION ACCORDINGLY.

STEP 1: ANALYZE YOUR SITE ON SECURITYHEADERS.COM

VISIT [SECURITYHEADERS.COM](https://securityheaders.com) AND ENTER YOUR SITE'S DOMAIN. THE TOOL GRADES YOUR HTTP RESPONSE HEADERS AND REPORTS WHICH ESSENTIAL HEADERS ARE MISSING OR MISCONFIGURED. IT CHECKS FOR HEADERS LIKE **CSP**, **HSTS**, **X-CONTENT-TYPE-OPTIONS**, **REFERRER-POLICY**, AND OTHERS.[DATADOME](https://datadome.com)

STEP 2: ENABLE APACHE'S HEADERS MODULE

BEFORE ADDING SECURITY HEADERS, ENSURE THAT MOD_HEADERS IS ENABLED:

```
BASH
```

```
SUDO A2ENMOD HEADERS
```

```
SUDO SYSTEMCTL RESTART APACHE2
```

THIS MODULE ALLOWS APACHE TO SET AND MANAGE CUSTOM HTTP HEADERS.[INFOSECWRITEUPS](https://infosecwriteups.com)

STEP 3: ADD SECURITY HEADERS IN APACHE

EDIT YOUR APACHE CONFIGURATION FILE (/ETC/HTTPD/CONF/HTTPD.CONF OR /ETC/APACHE2/SITES-AVAILABLE/000-DEFAULT.CONF) AND ADD THE FOLLOWING LINES INSIDE THE <VIRTUALHOST *:80> OR <VIRTUALHOST *:443> BLOCK
:[STUDYTONIGHT+1](https://studytonight.com)

```
TEXT
```

```
# XSS PROTECTION
```

HEADER ALWAYS SET X-XSS-PROTECTION "1; MODE=BLOCK"

PREVENT MIME-SNIFFING

HEADER ALWAYS SET X-CONTENT-TYPE-OPTIONS "NOSNIFF"

CLICKJACKING PROTECTION

HEADER ALWAYS SET X-FRAME-OPTIONS "SAMEORIGIN"

CONTENT SECURITY POLICY (CSP) - ADJUST FOR YOUR SITE

HEADER ALWAYS SET CONTENT-SECURITY-POLICY "DEFAULT-SRC 'SELF'; SCRIPT-SRC 'SELF' HTTPS://TRUSTED-SCRIPTS.COM; STYLE-SRC 'SELF' HTTPS://TRUSTED-STYLES.COM"

REFERRER POLICY

HEADER ALWAYS SET REFERRER-POLICY "STRICT-ORIGIN-WHEN-CROSS-ORIGIN"

ENFORCE HTTPS VIA HSTS

HEADER ALWAYS SET STRICT-TRANSPORT-SECURITY "MAX-AGE=31536000; INCLUDESUBDOMAINS; PRELOAD"

PERMISSIONS POLICY (OPTIONAL MODERN HEADER)

HEADER ALWAYS SET PERMISSIONS-POLICY "GEOLOCATION=(), CAMERA=(), MICROPHONE=()"

AFTER SAVING, RESTART APACHE:

BASH

SUDO SYSTEMCTL RESTART APACHE2

STEP 4: VERIFY YOUR CONFIGURATION

RE-SCAN YOUR SITE AT **SECURITYHEADERS.COM** AFTER MAKING THESE CHANGES. THE SITE SHOULD NOW RETURN AN IMPROVED GRADE (A OR A+) AND ALL CRITICAL HEADERS SHOULD BE VISIBLE IN THE SCAN RESULTS.[INFOSECWRITEUPS+1](#)

SUMMARY OF HEADERS AND THEIR PURPOSE

HEADER	PURPOSE
X-CONTENT-TYPE-OPTIONS	PREVENTS MIME-TYPE SNIFFING
X-FRAME-OPTIONS	MITIGATES CLICKJACKING
X-XSS-PROTECTION	ENABLES BROWSER XSS FILTERS
CONTENT-SECURITY-POLICY	MITIGATES XSS BY RESTRICTING CONTENT SOURCES
STRICT-TRANSPORT-SECURITY	ENFORCES USE OF HTTPS FOR SECURE CONNECTIONS
REFERRER-POLICY	CONTROLS REFERRER INFORMATION SENT WITH REQUESTS
PERMISSIONS-POLICY	LIMITS ACCESS TO BROWSER FEATURES LIKE CAMERA, GEOLOCATION, AND MIC

ONCE IMPLEMENTED, VERIFY THAT HEADERS ARE SENT CORRECTLY USING:

BASH